

Mathematics 4MB3/6MB3 Mathematical Biology

<http://ms.mcmaster.ca/earn/4MB3>

Fall 2025 ASSIGNMENT 2

This assignment is due on [Crowdmark](#) on **Wednesday 8 October 2025 at 11:59pm**. Do not submit a hardcopy. In addition, a compressed archive of all your source files must be e-mailed to the instructor and TA (unless you choose to invite the instructor and TA to an associated [github](#) repository).

General Notes

- (i) **Please re-read all the [General Notes](#) listed for Assignment 1!!**
- (ii) **Please re-read all the [Technical Comments](#) listed for Assignment 1!!**

New Technical Comments (read carefully!)

- (i) Explain your logic in computer code using embedded comments. The comment character in $\text{\LaTeX}$ is `%`. The comment character in $\text{\R}$ is `#`.
- (ii) Any graphics must be created in $\text{\R}$. Once you get the hang of it, the easiest way to combine $\text{\R}$ with $\text{\LaTeX}$ is to use the `knitr` package. You are encouraged to use `knitr`, but if you prefer you can create graphics separately and input them as `pdf` files into $\text{\LaTeX}$. (Note that you will be required to use `knitr` for the final project.)
- (iii) A note on importing $\text{\R}$ graphics into $\text{\LaTeX}$: When you run an $\text{\R}$ script in `RStudio`, any plots are shown by default in the bottom right pane of the `RStudio` window. When you are developing the code to make a plot, that is usually what you want. But in order to get the plot into a $\text{\LaTeX}$ document you must save the plot as a `pdf` file instead. In order to save a plot into the file `mylovelyplot.pdf`, do the following

```
pdf("mylovelyplot.pdf")
#### INSERT PLOTTING CODE HERE ####
dev.off()
```

The `pdf()` command changes the graphics output device to the named pdf file. This command has various optional arguments, such as `width` and `height`, which you may well want to use (rather than accepting the default width and height). The closing command `dev.off()` shuts off the current graphics output device, which means that the pdf file be complete. If you forget `dev.off()` then your pdf viewer will complain that the file you are trying to view is corrupt. Once you have created the required pdf file, to include it in your L^AT_EX document you can use the following command at the point where you want the plot:

```
\includegraphics{mylovelyplot.pdf}
```

Often, you need to control the size of the plot in your document (which is done with L^AT_EX's `\scalebox{}` command) and frequently you will want graphs to appear as figures with captions (which is done using the L^AT_EX `figure` environment). I also recommend putting all included graphics files into an `images` subfolder rather than cluttering the main folder where you are working. Here's how to implement all these things:

```
\begin{figure}
  \begin{center}
    \scalebox{0.5}{
      \includegraphics{images/mylovelyplot.pdf}
    }
  \end{center}
  \caption{This lovely plot is really inspirational for me.}
  \label{F:mylovelyplot}
\end{figure}
```

Note that I've added a few more details above: I made sure the plot will be centred using the `center` environment and I included a caption using the `\caption{}` command. I also created a label for the figure, the purpose of which is to allow us to refer to this figure by number without knowing what the number is. For example, in your L^AT_EX document you might say

my lovely plot is shown in Figure~\ref{F:mylovelyplot}

which will appear in the typeset version as “my lovely plot is shown in Figure 2” (assuming the figure in question is currently the second figure; if you reorder the figures in your document, L^AT_EX rennumbers everything for you).

Note: For those of you who have chosen to use the `knitr` package, you do not want to use the `pdf()` command. `knitr` takes care of the graphics file generation for you.

Finally, if you want to get really fancy (*i.e.*, publication-quality graphics), then rather than the `pdf` device you can use the `tikz` device, which understands L^AT_EX code in character strings. You will first need to install the `tikzDevice` package. Something that seems to catch everyone who uses `tikz` is that backslashes must be escaped in strings. Thus, for example, if you want $\sum_n \sin n\theta$ to appear on your plot, the character string you need in your R code is `"$\\sum_n\\sin{n}\\theta$"` (note the double backslashes!). To use `tikz` in ordinary R code you would typically use this structure:

```
library("tikzDevice")
tikz("mygraph.tex",standAlone=TRUE)
#### GRAPHICS CODE HERE ####
dev.off()
```

and then you would need to run `mygraph.tex` through L^AT_EX to produce the desired `pdf` file. If you are using `knitr` then you need the `tikz` chunk option (`dev="tikz"`).

1 Plot P&I mortality in Philadelphia in 1918

- (a) Confirm that you have received this data file by e-mail:

`pim.us.phila.city.1918.dy.csv`

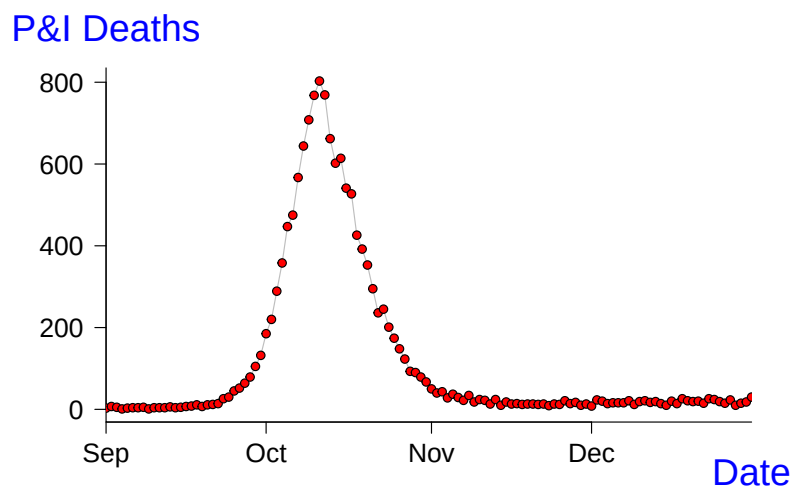
This plain text comma-separated-value file can be examined (if you wish) using any plain text editor, such as `Emacs`.

- (b) Read the data into a data frame in , using the `read.csv()` function. For example, the following chunk of  code should work:

```
datafile <- "pim_us_phila_city_1918_dy.csv"
philadata <- read.csv(datafile)
philadata$date <- as.Date(philadata$date)
```

The purpose of the last line of code above is to ensure that  encodes character strings such as "1918-10-15" as dates.

- (c) Reproduce the Philadelphia 1918 P&I plot:



You'll need to use functions such as `plot()`, `points()` and `lines()`. For a comprehensive list of graphics parameters accepted by these functions, enter `?par` into the Console pane in RStudio. There are multiple ways to produce a graph exactly like the above, but the following steps work:

- Use `plot()` to draw the box and basic annotation and the grey line. Suppress labels when doing this (*e.g.*, `xlab=""`). The box type is controlled by the `bty` option and the orientation of annotation is controlled by the `las` option.

- Use `points()` to draw the heavy red dots with black borders. The most elegant way to do this is to set the point character type to 21 (`pch=21`) and the point background colour to red (`bg="red"`). Alternatively, you can use `points()` twice (first to draw the red dots and then to draw the black circles around them).
- Use `mtext()` to add the x and y axis labels in the margins of the plot.

2 Estimate $\mathcal{R}_0$ from the Philadelphia P&I time series

- The observed mortality time series $M(t)$ is certainly not equal to the prevalence $I(t)$ that appears in the SIR model. Suppose, however, that $I(t) = \eta M(t - \tau)$ for all time (where η and τ are constants), *i.e.*, that the mortality curve is exactly a scaled and translated version of the prevalence curve. Prove that if both I and M are growing exactly exponentially over some time period then their exponential rates are identical. Thus, if we compare them during the “exponential phase” on a logarithmic scale, then both curves will be perfectly straight with exactly the same slope.
- Fit a straight line to the part of the Philadelphia 1918 mortality time series that looks straight on a logarithmic scale (and show your result in a plot). Once you get the hang of it, the easiest way to do this is to use the `lm()` function in  (`lm` stands for linear model). Note that the simplest way to draw a straight line with given slope and intercept is with the `abline()` function. If you find `lm()` counter-intuitive to understand then experiment with `abline()` until your eyes tell you that you have discovered a line that provides a good fit.
- How is the slope of your fitted line related to the parameters of the SIR model? (*Hint:* When I is small, $S \simeq 1$.) Why do you need an independent measure of the mean infectious period to estimate $\mathcal{R}_0$? If the mean infectious period is 4 days, what is your estimate of $\mathcal{R}_0$?

3 Fit the basic SIR model to the Philadelphia P&I time series

- (a) Install the "deSolve" package. This is done by typing the following command in the Console pane of RStudio:

```
install.packages("deSolve")
```

You will then be prompted to choose a mirror site from which to download the package. It doesn't matter which mirror you choose, but choosing a site in Ontario might save a fraction of a second. *Note:* This is a one-time operation. You do not want an `install.packages()` command inside your solutions code.

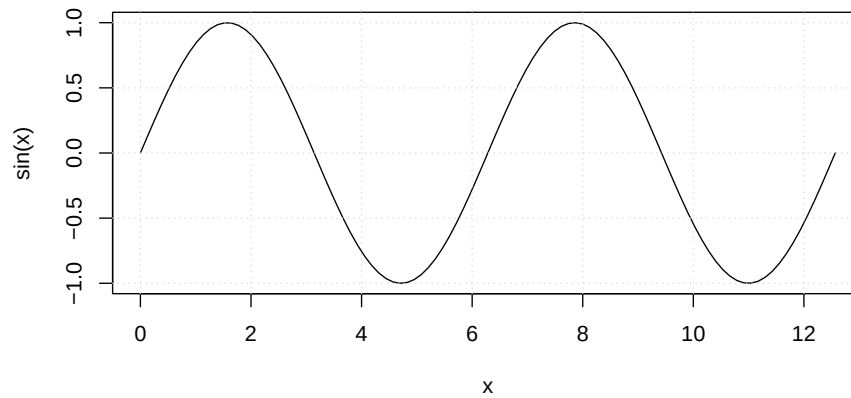
- (b) Write an  function that plots the solution $I(t)$ of the SIR model for given parameter values ($\mathcal{R}_0$ and $1/\gamma$) and given initial conditions (S_0, I_0). Use the `ode()` function in the `deSolve` package. A few hints:

- Your code will first need to load the `deSolve` package:

```
library("deSolve")
```

- As an example of defining a function (without getting involved with a differential equation), here is a code chunk that defines a function to plot a sine curve, and then executes the function. Note that the default min and max x values are set in the parameter list of the function definition, but the max x value is changed when the function is executed:

```
plot.sine <- function( xmin=0, xmax=2*pi ) {  
  x <- seq(xmin,xmax,length=100)  
  plot(x, sin(x), typ="l")  
  grid() # add a light grey grid  
}  
plot.sine(xmax=4*pi)
```



- Here's another example. This time we first define the vector field for a differential equation. We then use this function inside another function that plots the solution of the associated differential equation. To understand the construction, you can, as usual, study the help page for the calling function (`?ode` in this case), but the most important issues are the following.

One of the arguments of the `ode()` function is the function that evaluates the vector field at the current time. To avoid confusion, choose the arguments of your vector field function to be `t`, `vars` and `parms` (in that order):

- `t` The current time, which will be used within the vector field function if the system is non-autonomous.
- `vars` A named vector of the variables in the system (*e.g.*, S , I). The variables, as named vector passed to this function, are used in the code that defines the vector field within the function.
- `parms` A named vector of the parameters of the system (*e.g.*, β , γ). It is convenient—but not necessary—to specify default values for the parameters.

It is strongly recommended that you follow exactly the style below when defining vector fields for differential equations that you wish

to solve with the `ode()` function. In particular, the construction “`with(as.list(c(parms,vars)), ...)`” makes the variables and parameters visible within the section of code between the braces (`{...}`) without having to refer to the vectors or lists in which they are stored. For example, the code would be much harder to read if each instance of `x` were replaced by `vars$x` and each instance of `beta` were replaced by `parms$beta`; this issue becomes extremely important for complicated vector fields.

```
## Vector Field for SI model
SI.vector.field <- function(t, vars, parms=c(beta=2,gamma=1)) {
  with(as.list(c(parms, vars)), {
    dx <- -beta*x*y # dS/dt
    dy <- beta*x*y   # dI/dt
    vec.fld <- c(dx=dx, dy=dy)
    return(list(vec.fld)) # ode() requires a list
  })
}
```

The following function plots a single solution of the ODE for a given initial condition (`ic`), integration time (`tmax`) and times at which the state is to be returned (`times`). The vector field function is passed as the `func` argument and the parameter vector is passed as the `parms` argument. If further arguments are given, they are passed to the `lines()` function that draws the solution.

```
## Draw solution
draw.soln <- function(ic=c(x=1,y=0), tmax=1,
                      times=seq(0,tmax,by=tmax/500),
                      func, parms, ... ) {
  soln <- ode(ic, times, func, parms)
  lines(times, soln[, "y"], col="blue", lwd=3, ... )
}
```

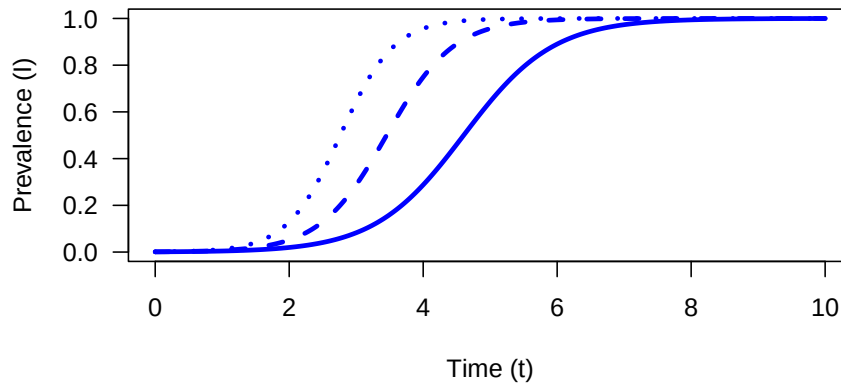
Note here that the call to the `ode()` function gives the arguments in the default order so they are interpreted correctly. If we wished to write the arguments in a different order then we would have to be explicit about which argument is which. For example, if we wanted

to list the initial conditions last for some deep reason then we would have to write:

```
soln <- ode(times=times, func=func, parms=parms, y=ic)
```

We can now use our `draw.soln()` function to plot a few solutions of the SI model.

```
## Plot solutions of the SI model
tmax <- 10 # end time for numerical integration of the ODE
## draw box for plot:
plot(0,0,xlim=c(0,tmax),ylim=c(0,1),
     type="n",xlab="Time (t)",ylab="Prevalence (I)",las=1)
## initial conditions:
I0 <- 0.001
S0 <- 1 - I0
## draw solutions for several values of parameter beta:
betavals <- c(1.5,2,2.5)
for (i in 1:length(betavals)) {
  draw.soln(ic=c(x=S0,y=I0), tmax=tmax,
            func=SI.vector.field,
            parms=c(beta=betavals[i],gamma=1),
            lty=i # use a different line style for each solution
  )
}
```



- (c) For $I_0 = 10^{-3}$ and $S_0 = 1 - I_0$, plot the solutions of the SIR model assuming $1/\gamma = 4$ days and $\mathcal{R}_0 \in \{1.2, 1.5, 1.8, 2, 3, 4\}$. Use the `legend()` command to make a legend on the plot that shows which curves correspond to which values of $\mathcal{R}_0$.
- (d) By trial and error, find values of $\mathcal{R}_0$ and γ that yield a solution of the SIR model that fits the Philadelphia P&I times series reasonably well. You can assess the quality of fit using the Euclidean distance between the model solution and the data. (*Note:* The trial and error approach is a valuable exercise, but not a suggestion of a method you would really use in practice. We'll discuss better methods for fitting ODE models to data later.)

4 Executive summary for the Public Health Agency

The Public Health Agency of Canada (PHAC) is revising their pandemic plan and has asked your group to summarize what you learned from analyzing the 1918 Philadelphia P&I time series. Besides explaining what inferences you feel you can make from your analysis so far, PHAC wants to know what you would investigate if they were to fund you to continue your work full time

for a month. They want a maximum of one page from your group.

Incidentally, you might be interested to know that rumour has it that all of the members of the pandemic planning committee took Math 2C03 at McMaster University between 1980 and 2003, but they all failed. Also, when the chair of the committee was recently asked “What is a differential equation?” he apparently bent over and vomited (it is hard to know quite what to make of this given that PHAC was investigating a norovirus outbreak at the time).

Note: When submitting your assignment solution, it is imperative that the one-page executive summary be printed on its own page. To start a new page in $\text{\LaTeX}$, use the `\newpage` command. Also, as usual, your summary should be in 12 point font. Don't try to cram in as much as possible. Make that page as clear and concise as you can, so that a public health planner can absorb its content quickly and easily.

— END OF ASSIGNMENT —

Compile time for this document: September 25, 2025 @ 16:03